

Framework móvil para un negocio local de Santa Cruz

Caso de aplicación: PetCare Santa Cruz (veterinaria) – Proyecto ConectaCruz

Rodrigo Calle Cruz

Jhery León Carballo

Karen Mamani Salas

Docente

Ing. Requena Llorentty Jimmy Nataniel

Materia

Aplicaciones Móviles I

Actividad 01 (grupal) – Frameworks móviles

Link del Repositorio: https://github.com/AM1-LosMosqueperris/Mosqueperris.git

Santa Cruz de la Sierra - noviembre de 2026

INDICE

1. Introducción 4
1.1 Contexto 4
1.2 Problema de decisión y pregunta de investigación 4
1.3 Objetivos 4
1.4 Evidencia publicada y supuestos 5
2. Métodos 6
2.1 Diseño y fuentes 6
2.2 Alternativas comparadas 6
2.3 Criterios, ponderaciones y escala 7
2.4 Entorno de implementación 8
3. Resultados 8
3.1 Matriz comparativa y puntajes 8
3.2 Resultado ponderado 9
3.3 Evidencia de implementación 10
4. Discusión 12
4.1 Qué respalda la elección 12
4.2 Qué contradice o debilita la elección 13
4.3 Límites 13
4.4 Efecto en el desarrollo de PetCare y ConectaCruz 14
4.5 Contraste de la decisión con el equipo 14
5. Conclusiones 15
Referencias 16

Resumen

Esta investigación determina qué framework móvil conviene a un emprendimiento local de Santa Cruz de la Sierra que necesita catálogo, pedidos y reservas con presupuesto limitado. El caso de aplicación es PetCare Santa Cruz, una veterinaria que ofrece servicios con cita previa y registro de mascotas, desarrollada dentro del proyecto Conecta Cruz. Se realizó una comparación documental de Flutter, React Native y desarrollo nativo (Kotlin y Swift) con cinco criterios ponderados: costo de desarrollo (25 %), rendimiento (20 %), comunidad y soporte (15 %), compatibilidad con Android e iOS (25 %) y curva de aprendizaje (15 %). Los puntajes ponderados fueron 4,35 para Flutter, 4,20 para React Native y 3,05 para el desarrollo nativo, por lo que se propone Flutter. Para comprobar la viabilidad se instaló el entorno y se construyó un prototipo funcional con pantalla de bienvenida y registro de mascotas, ejecutado en un emulador Pixel 7 y en un teléfono Android real. La diferencia entre Flutter y React Native es pequeña y los puntajes son valoraciones del equipo, no mediciones propias; la elección debe confirmarse con pruebas de rendimiento, pruebas en iOS y pruebas con usuarios.

Palabras clave: framework móvil, Flutter, React Native, desarrollo nativo, emprendimiento local, PetCare Santa Cruz.

1. Introducción

1.1 Contexto

Muchos comercios de barrio en Santa Cruz de la Sierra atienden a sus clientes por WhatsApp y redes sociales: publican fotos, responden consultas y coordinan pedidos o citas por mensaje. Este es el escenario que se asume para el proyecto, no una cifra comprobada con una fuente publicada. Estos canales son accesibles y no requieren inversión inicial, pero cuando crece la cantidad de clientes la información queda repartida entre chats, llamadas y notas, y el propietario puede perder una solicitud o asignar el mismo horario dos veces.

PetCare Santa Cruz es una veterinaria que necesita justamente organizar tres procesos: mostrar su catálogo de servicios con precios (consulta general, vacunación, desparasitación, peluquería, control y laboratorio), registrar a las mascotas de sus clientes y recibir solicitudes de cita. Una aplicación móvil puede reunir estos procesos en una sola interfaz, pero antes de construirla hay que decidir con qué tecnología se desarrollará.

1.2 Problema de decisión y pregunta de investigación

Elegir el framework por preferencia personal puede aumentar el tiempo de desarrollo, exigir conocimientos difíciles de conseguir o duplicar el trabajo para Android e iOS. La pregunta que guía el trabajo es: ¿Qué framework móvil conviene a un emprendimiento local de Santa Cruz que necesita catálogo, pedidos y reservas con un presupuesto limitado?

1.3 Objetivos

Objetivo general. Elegir y justificar el framework móvil más conveniente para una aplicación de catálogo, pedidos y reservas de un negocio local, tomando como caso PetCare Santa Cruz.

Objetivos específicos:

- Comparar Flutter, React Native y desarrollo nativo según costo, rendimiento, comunidad, compatibilidad con Android e iOS y curva de aprendizaje.

- Proponer una elección con criterios ponderados y fuentes oficiales y académicas.
- Comprobar la propuesta instalando Flutter y construyendo la primera pantalla de la aplicación PetCare, ejecutada en emulador y en un teléfono real.

1.4 Evidencia publicada y supuestos

La Tabla 1 separa lo que afirman las fuentes consultadas de lo que son supuestos o valoraciones del equipo.

Tabla 1

Evidencia publicada frente a supuestos del equipo

Afirmación	Tipo	Fuente
Flutter permite construir aplicaciones para varias plataformas desde una base de código común.	Evidencia (declaración oficial)	Google (s. f.)
React Native construye interfaces con React y componentes nativos de cada plataforma.	Evidencia (declaración oficial)	Meta (s. f.)
El desarrollo nativo usa herramientas y lenguajes propios de cada sistema (Android Studio y Kotlin; Swift para iOS).	Evidencia (declaración oficial)	Android Developers (s. f.); Apple (s. f.)
En benchmarks sobre Android, Flutter suele imponer la menor sobrecarga de tiempo y energía y React Native la mayor; en una prueba con animación continua, React Native usó menos CPU y energía.	Evidencia académica (con límites)	Oliveira et al. (2023)

En dos aplicaciones equivalentes medidas en un Huawei P20 Lite, los resultados no indican claramente cuál tecnología es más eficiente.	Evidencia académica (con límites)	Markowski y Smołka (2023)
Los clientes de una veterinaria barrial ya coordinan citas por WhatsApp y redes.	Supuesto del equipo	No verificado
Compartir código entre Android e iOS reduce el esfuerzo del proyecto.	Supuesto razonable, no medido	Sin medición propia
Los pesos y puntajes de la matriz de decisión.	Valoración del equipo	Elaboración propia

Nota. Elaboración propia. Las fuentes oficiales describen características declaradas por sus responsables; no son mediciones independientes.

2. Métodos

2.1 Diseño y fuentes

Se realizó una investigación documental, descriptiva y comparativa. Se usaron las páginas oficiales de Flutter, React Native, Android Developers y Apple Developer para describir lenguajes y herramientas, y dos estudios académicos sobre uso de recursos (Oliveira et al., 2023; Markowski y Smołka, 2023) para reconocer que el rendimiento depende de la aplicación, el dispositivo y las condiciones de prueba. Las páginas oficiales se consultaron el 7 de noviembre de 2026.

2.2 Alternativas comparadas

Flutter. Conjunto de herramientas de Google que usa el lenguaje Dart y permite crear aplicaciones para varias plataformas desde una base de código compartida (Google, s. f.). Construye la interfaz con widgets y ofrece Hot Reload para ver cambios sin reiniciar la aplicación en muchos casos.

React Native. Permite desarrollar aplicaciones móviles con React y JavaScript o TypeScript, usando componentes que se corresponden con elementos nativos (Meta, s. f.). Resulta cercano a equipos con experiencia en desarrollo web.

Desarrollo nativo. Una aplicación distinta para cada sistema: Android Studio y Kotlin en Android (Android Developers, s. f.) y Swift en iOS (Apple, s. f.). Da control directo de la plataforma, pero obliga a mantener dos implementaciones y dos conjuntos de conocimientos.

2.3 Criterios, ponderaciones y escala

Los criterios salen de la pregunta de investigación y de las funciones de PetCare (catálogo de servicios, registro de mascotas y solicitud de citas). Se dio más peso al costo y a la compatibilidad con Android e iOS porque el presupuesto es limitado y el negocio debe llegar a ambos tipos de teléfono. Cada alternativa se puntuó de 1 (muy desfavorable) a 5 (muy favorable).

Tabla 2

Criterios de comparación y ponderaciones

Criterio	Peso	Qué considera
Costo de desarrollo	25 %	Tiempo, reutilización de código y esfuerzo de mantenimiento.
Rendimiento	20 %	Fluidez, respuesta de la interfaz y uso de recursos.
Comunidad y soporte	15 %	Documentación, ejemplos, paquetes y ayuda disponible.
Compatibilidad Android/iOS	25 %	Esfuerzo para ofrecer las mismas funciones en ambos sistemas.
Curva de aprendizaje	15 %	Tiempo para que el equipo trabaje con la tecnología.

Nota. Los pesos son decisiones metodológicas del equipo; no provienen de los fabricantes ni de los estudios consultados.

La puntuación ponderada se calcula como P = Σ (calificación × peso), con los pesos expresados como decimales (suman 1,00). Por ejemplo, para Flutter: (5 × 0,25) + (4 × 0,20) + (4 × 0,15) + (5 × 0,25) + (3 × 0,15).

2.4 Entorno de implementación

Para comprobar la propuesta se instaló Flutter junto con Android Studio y su SDK, se creó un emulador Pixel 7 (API 37) y se ejecutó flutter doctor para revisar el entorno. La aplicación se probó además en un teléfono Android conectado por cable USB. No se dispuso de un dispositivo iOS.

3. Resultados

3.1 Matriz comparativa y puntajes

Tabla 3

Puntajes por criterio y alternativa (escala de 1 a 5)

Criterio	Peso	Flutter	React Native	Nativo
Costo de desarrollo	25 %	5	4	2
Rendimiento	20 %	4	3	5
Comunidad y soporte	15 %	4	5	5
Compatibilidad Android/iOS	25 %	5	5	2
Curva de aprendizaje	15 %	3	4	2

Nota Elaboración propia a partir de la documentación oficial y de los estudios consultados. Los puntajes son estimaciones del equipo para el caso PetCare.

Tabla 4

Base de cada puntaje

Criterio	Base del puntaje
Costo	Flutter y React Native comparten una base de código para Android e iOS (Google, s. f.; Meta, s. f.); el nativo exige dos implementaciones. La diferencia de un punto entre Flutter y React Native es un supuesto del equipo.
Rendimiento	Flutter suele imponer menor sobrecarga y React Native mayor en los benchmarks de Oliveira et al. (2023); Markowski y Smołka (2023) no hallaron un ganador claro. El nativo conserva 5 por su acceso directo a la plataforma (supuesto).
Comunidad	Las tres alternativas cuentan con documentación oficial. El 5 de React Native y del nativo y el 4 de Flutter son supuestos no medidos.
Compatibilidad	Flutter y React Native apuntan a Android e iOS desde un mismo código; el nativo requiere aplicaciones separadas.
Curva de aprendizaje	Se supone que JavaScript y React son más conocidos que Dart; el nativo exige aprender Kotlin y Swift.

Nota. Los puntajes que no se apoyan en una fuente se marcan como supuesto.

3.2 Resultado ponderado

Tabla 5

Puntuación ponderada

Alternativa	Cálculo	Resultado
Flutter	1,25 + 0,80 + 0,60 + 1,25 + 0,45	4,35
React Native	1,00 + 0,60 + 0,75 + 1,25 + 0,60	4,20
Desarrollo nativo	0,50 + 1,00 + 0,75 + 0,50 + 0,30	3,05

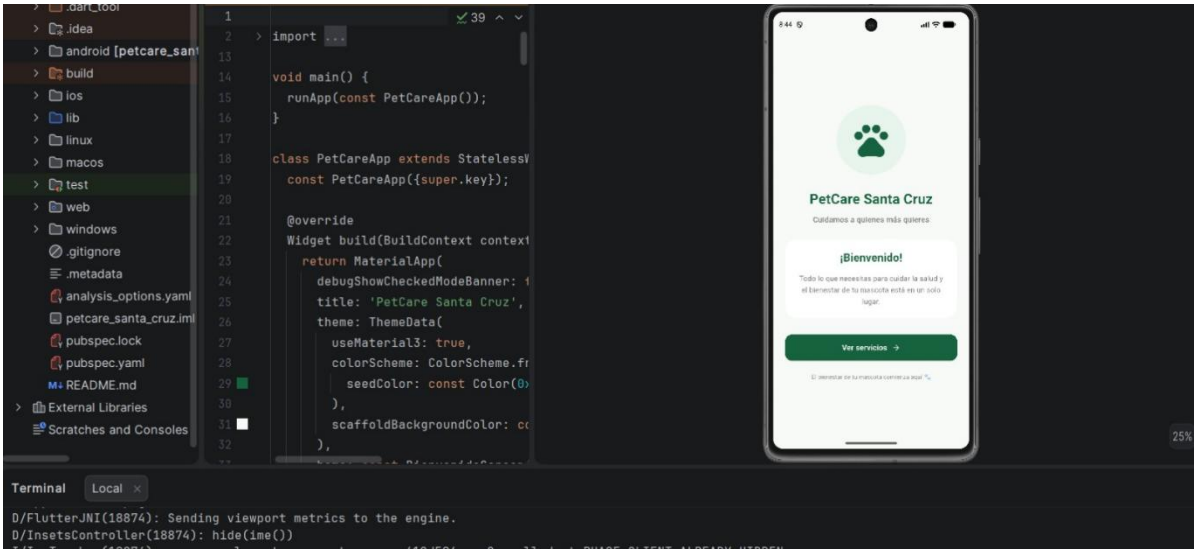
Elección propuesta: Flutter, con 4,35 puntos sobre 5.

3.3 Evidencia de implementación

La Figura 1 muestra la pantalla de bienvenida de PetCare Santa Cruz ejecutándose en el emulador Pixel 7 (API 37) desde Android Studio. Las Figuras 2 y 3 muestran la pantalla de bienvenida y la pantalla de registro de mascotas en un teléfono Android real conectado por USB. La Figura 4 presenta el resultado de flutter doctor.

Figura 1

Pantalla de bienvenida de PetCare Santa Cruz en el emulador Pixel 7 (API 37)



Nota. Captura propia en Android Studio. La pantalla muestra el título «PetCare Santa Cruz», la tarjeta «¡Bienvenido a PetCare!» con el botón «Reservar una cita», el resumen general y los accesos rápidos «Registrar mascota» y «Agendar consulta».

Figura 2

Pantalla de bienvenida en un teléfono Android real



Nota. Captura propia. La pantalla muestra el título «PetCare Santa Cruz», el mensaje «¡Bienvenido!» y el botón «Ver Servicios».

Figura 3

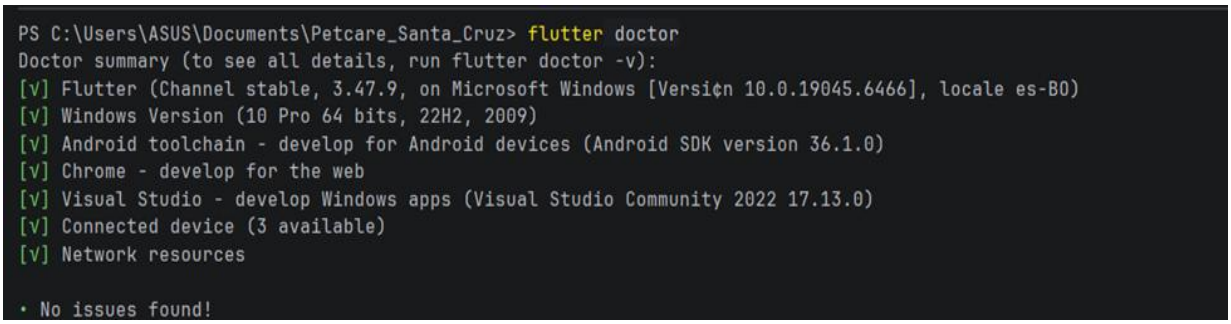
Pantalla de registro de mascotas en un teléfono Android real



Nota. Captura propia. Formulario con nombre de la mascota, especie, edad, raza y nombre del propietario.

Figura 4

Resultado de flutter doctor



Nota. Captura propia de la terminal de Android Studio. Flutter, Android toolchain (SDK 36.1.0), Chrome, dispositivos conectados (3), Visual Studio y recursos de red aparecen sin errores.

Repositorio del proyecto: https://github.com/AM1-LosMosqueperris/Mosqueperris.git

Video de presentación y defensa: Video Presentación .mp4

Estructura de la pantalla de bienvenida: la aplicación parte de PetCareApp, un widget sin estado que devuelve un MaterialApp con Material 3 y un esquema de color verde generado desde una semilla. Su pantalla principal es PetCareNavigation, que organiza la barra de título, el contenido de cada sección y la barra de navegación inferior con cinco destinos: Inicio, Mascotas, Servicios, Citas y Vacunas. La pantalla Inicio contiene una tarjeta de bienvenida con un botón que lleva a la reserva de cita, tres tarjetas de resumen y una lista de accesos rápidos.

4. Discusión

4.1 Qué respalda la elección

- Flutter obtuvo el mayor puntaje ponderado (4,35), por encima de React Native (4,20) y del desarrollo nativo (3,05).
- Oliveira et al. (2023) observaron que, en general, Flutter impone la menor sobrecarga de tiempo de ejecución y energía entre los frameworks multiplataforma que evaluaron en Android.

- Una sola base de código cubre Android e iOS, algo importante con presupuesto limitado.
- El prototipo de PetCare ya funciona: se ejecutó en un emulador y en un teléfono real, con formularios, tarjetas, listas y navegación inferior construidos con widgets reutilizables.

4.2 Qué contradice o debilita la elección

- React Native supera a Flutter en comunidad y en curva de aprendizaje; entre ambos suman 0,30 puntos de ventaja que Flutter compensa con 0,45 puntos en costo y rendimiento. La diferencia final es de solo 0,15 puntos.
- Markowski y Smołka (2023) no encontraron una tecnología más eficiente que otra, y Oliveira et al. (2023) identificaron un caso en que React Native usó menos CPU y energía. Por eso el rendimiento no puede darse por resuelto.
- Los estudios midieron benchmarks y aplicaciones concretas en Android, no una aplicación de citas veterinarias, y no se hicieron mediciones propias.
- Dart es un lenguaje nuevo para el equipo; el costo de aprenderlo puede ser mayor al supuesto.

4.3 Límites

- Solo se probó en un emulador y en un teléfono Android; no se probó en iOS ni en varios modelos de dispositivo.
- No se construyó un prototipo equivalente en React Native, por lo que la comparación es documental.
- No se midió tiempo de inicio, consumo de memoria ni energía.
- No se estimaron costos locales de desarrollo ni se hicieron pruebas con usuarios.
- Los pesos y varios puntajes son valoraciones del equipo.
- flutter doctor reporta una advertencia en Visual Studio (componentes de escritorio Windows). No afecta a Android, que es el destino de la aplicación, pero queda pendiente de resolver si se quisiera compilar para Windows.

4.4 Efecto en el desarrollo de PetCare y ConectaCruz

Elegir Flutter permite continuar con el prototipo ya construido y ampliarlo por módulos: primero el registro de mascotas y el catálogo de servicios, luego la reserva de citas con fecha y hora, y después vacunas e historial. Para que los datos se guarden y el propietario pueda administrarlos se necesitará un servicio de datos o una API, y perfiles con permisos diferenciados entre cliente y veterinaria. Como siguientes validaciones se proponen medir el rendimiento en más teléfonos, probar en iOS si se consigue un dispositivo y pedir a algunos clientes que registren una mascota y soliciten una cita**.**

4.5 Contraste de la decisión con el equipo

Antes de fijar la elección, cada integrante propuso la opción que consideraba mejor para PetCare Santa Cruz y explicó sus razones. Luego el equipo comparó esos argumentos con los puntajes de la Tabla 3. La Tabla 6 resume la postura inicial de cada uno.

Tabla 6

Postura inicial de cada integrante

Integrante	Opción que propuso	Razón principal
Rodrigo J. Calle Cruz	Flutter	Un solo código para Android e iOS
Jhery León	React Native	Ya conoce JavaScript
Karen Mamani	Flutter	Los widgets facilitan el diseño

Nota. Registro del equipo.

Decisión acordada: el equipo seleccionó Flutter por obtener la mayor puntuación en la matriz (4,35) y por permitir una sola base de código para Android e iOS.

5. Conclusiones

Con los criterios y pesos definidos, Flutter obtuvo la mayor puntuación (4,35) para una aplicación de catálogo, citas y registro como PetCare Santa Cruz, seguido por React Native (4,20) y el desarrollo nativo (3,05). Se elige Flutter por su base de código compartida para Android e iOS, por la evidencia de menor sobrecarga en los benchmarks de Oliveira et al. (2023) y porque el prototipo ya se ejecuta en un emulador y en un teléfono real.

La decisión es una propuesta fundamentada, no una prueba definitiva: la ventaja sobre React Native es pequeña y se basa en valoraciones del equipo. Antes de ampliar el proyecto conviene medir el rendimiento, probar en iOS y validar la aplicación con usuarios.

Referencias

Android Developers. (s. f.). *Android documentation*. Recuperado el 7 de noviembre de 2026, de https://developer.android.com/docs

Apple. (s. f.). *Swift*. Apple Developer. Recuperado el 7 de noviembre de 2026, de https://developer.apple.com/swift/

Google. (s. f.). *Flutter documentation*. Recuperado el 7 de noviembre de 2026, de https://docs.flutter.dev/

Markowski, M., y Smołka, J. (2023). A comparative analysis of resource use in the Flutter and React Native frameworks. *Journal of Computer Sciences Institute, 29*, 346–351. https://doi.org/10.35784/jcsi.3794

Meta. (s. f.). *React Native documentation*. Recuperado el 7 de noviembre de 2026, de https://reactnative.dev/docs/getting-started

Oliveira, W., Moraes, B., Castor, F., y Fernandes, J. P. (2023). Analyzing the resource usage overhead of mobile app development frameworks. En *Proceedings of the 27th International Conference on Evaluation and Assessment in Software Engineering* (pp. 152–161). Association for Computing Machinery. https://doi.org/10.1145/3593434.3593487